Project - High Level Design

On

Deploy Manufacturing App to Kubernetes

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Om Tiwari	EN22CS301669
02	Paridhi Shirwalkar	EN22CS301684
03	Nitesh Chourasiya	EN22CS301660
04	Mradul Jain	EN22CS301616

Group Name: 01D6

Project Number: AAI-12

Industry Mentor Name: Prof. Akshay Saxena

University Mentor Name:

Academic Year: 2025-2026

Table of Contents

1. Introduction.

    1.1. Scope of the document.
    1.2. Intended Audience
    1.3. System overview.

2. System Design.

    2.1. Application Design
    2.2. Process Flow.
    2.3. Information Flow.
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue.

3. Interfaces

4. State and Session Management

5. Caching

6. Functional Requirements

    6.1. User Authentication & Authorization
    6.2. Infrastructure Configuration Management
    6.3. Provisioning Control & Monitoring
    6.4. Dockerfile (Application Containerization)
    6.5. GitHub Actions (Workflow Configuration)
    6.6. CI/CD Strategy (Continuous Integration & Deployment)

7. Non-Functional Requirements

8. References

9. GitHub

1. Introduction

This document presents the system design for the **Deploy Manufacturing App to Kubernetes** project, developed as part of the Datagami Skill Based Course at Medicaps University. The project demonstrates a complete DevOps implementation involving containerization, cloud-native orchestration, and automated deployment of a Multi-Agent Manufacturing System built with Python, Streamlit, and CrewAI.

The application leverages Google Gemini LLM to power intelligent manufacturing agents. The DevOps infrastructure ensures the application is reliably packaged, deployed, and updated on a production-grade AWS EKS Kubernetes cluster with zero downtime on every release.

1.1 Scope of the Document

This document covers the complete system design of the deployment infrastructure for the Multi-Agent Manufacturing Streamlit application. Specifically, it includes:

- The overall system architecture and component interactions
- Docker containerization strategy and build process
- Kubernetes deployment and service configuration
- CI/CD pipeline design using GitHub Actions
- AWS EKS cloud infrastructure setup
- Environment variable and secrets management
- Security considerations and key design decisions
- API catalogue for internal application and health check endpoints

This document does **not** cover the internal AI logic of the CrewAI agents, the Gemini LLM integration details, or the business logic of the manufacturing domain. Those are addressed in the application-level documentation of the Multi-agent-system repository.

1.2 Intended Audience

This document is intended for the following readers:

Audience	Purpose
DevOps Engineers	Understand the deployment pipeline and infrastructure setup
Backend Developers	Understand how the application is containerized and served
Cloud Architects	Review the AWS EKS and networking design decisions
Academic Evaluators	Assess the technical depth and implementation of the project

Project Mentors	Review the overall system design for guidance and feedback
Future Maintainers	Onboard quickly and extend the infrastructure

A basic understanding of Docker, Kubernetes, GitHub Actions, and AWS services is recommended for reading this document.

1.3 System Overview

The system consists of two decoupled GitHub repositories working together to deliver a fully automated cloud deployment:

Repository 1- AI Application (Multi-agent-system) Contains the Streamlit frontend, CrewAI backend agents, task definitions, orchestrator logic, Pydantic schemas, and artifact storage. This repository holds all application-level code.

Repository 2- DevOps Infrastructure (Deploy-Manufacturing-App-to-Kubernetes) Contains the Dockerfile, Kubernetes manifests, GitHub Actions workflow, and dependency files. This repository holds no application code — it is purely infrastructure.

At a high level, whenever a developer pushes code to the DevOps repository, a GitHub Actions pipeline automatically builds a Docker image (which clones the AI application repo at build time), pushes the image to Docker Hub, and deploys it to an AWS EKS Kubernetes cluster. Kubernetes then performs a Zero-Downtime Rolling Update, ensuring the application remains available throughout the update process. The app is finally accessible to end users via an AWS Load Balancer public URL on port 80.

2. System Design

2.1 Application Design

The application follows a two-tier architecture consisting of a frontend UI layer and a backend agent layer, both packaged into a single Docker container and served via Streamlit.

Frontend Layer- frontend/app.py The Streamlit application serves as the user interface. Users interact with the app through their browser, entering manufacturing-related queries and providing a Google Gemini API key via the Streamlit sidebar. Streamlit listens on port 8501 inside the container and is bound to 0.0.0.0 to accept traffic from outside the container.

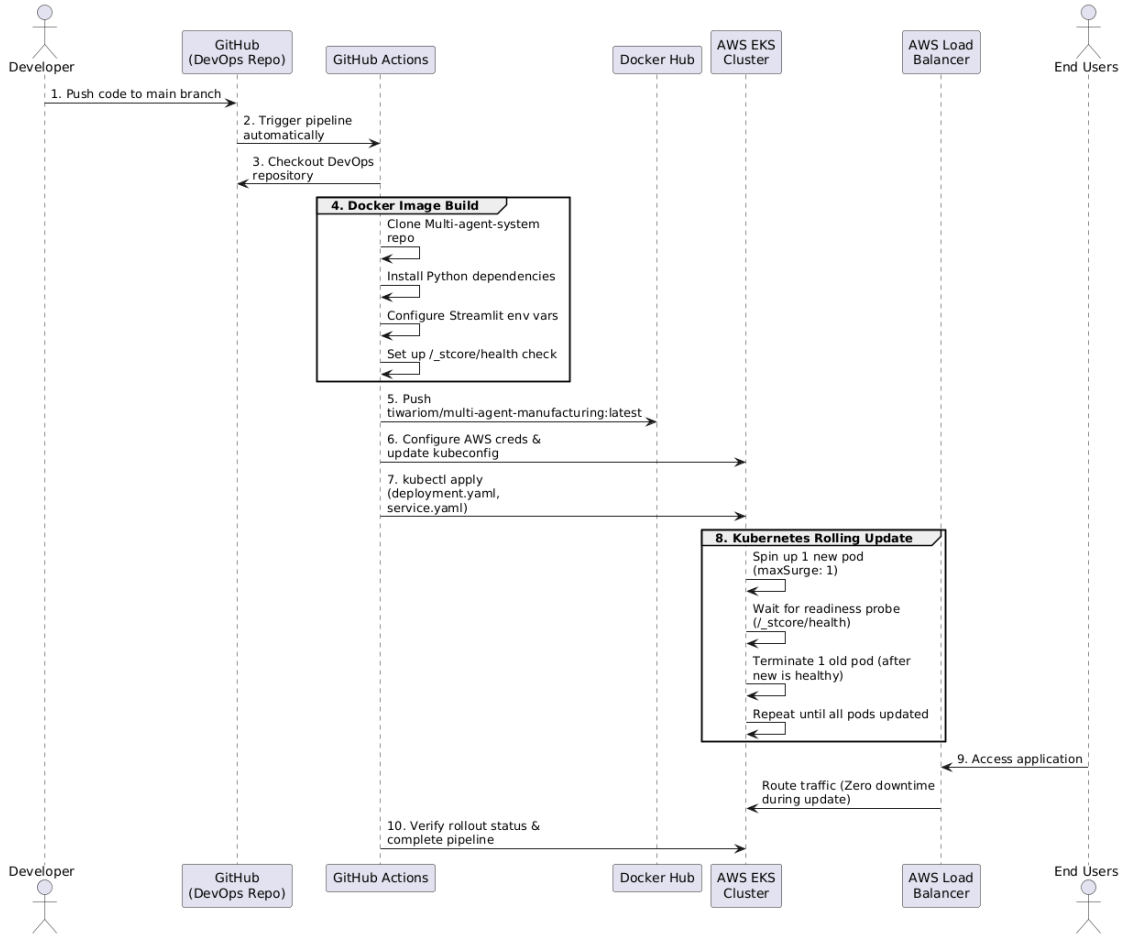
Backend Layer- backend/ The backend consists of the following modules:

Module	File	Responsibility
Agent Definitions	agents.py	Defines CrewAI agents with roles and goals
Workflow Controller	orchestrator.py	Coordinates agent execution and task flow
Task Definitions	tasks.py	Defines individual tasks assigned to agents
Data Validation	schemas.py	Pydantic models for input/output validation
Artifact Storage	storage.py	Manages saving and retrieving output artifacts

Container Design The entire application frontend and backend is packaged into a single Docker image using python:3.11-slim as the base. The Dockerfile clones the Multi-agent-system repository at build time, installs all dependencies from a clean Linux-compatible requirements.txt, and starts the Streamlit server as the container entry point.

2.2 Process Flow

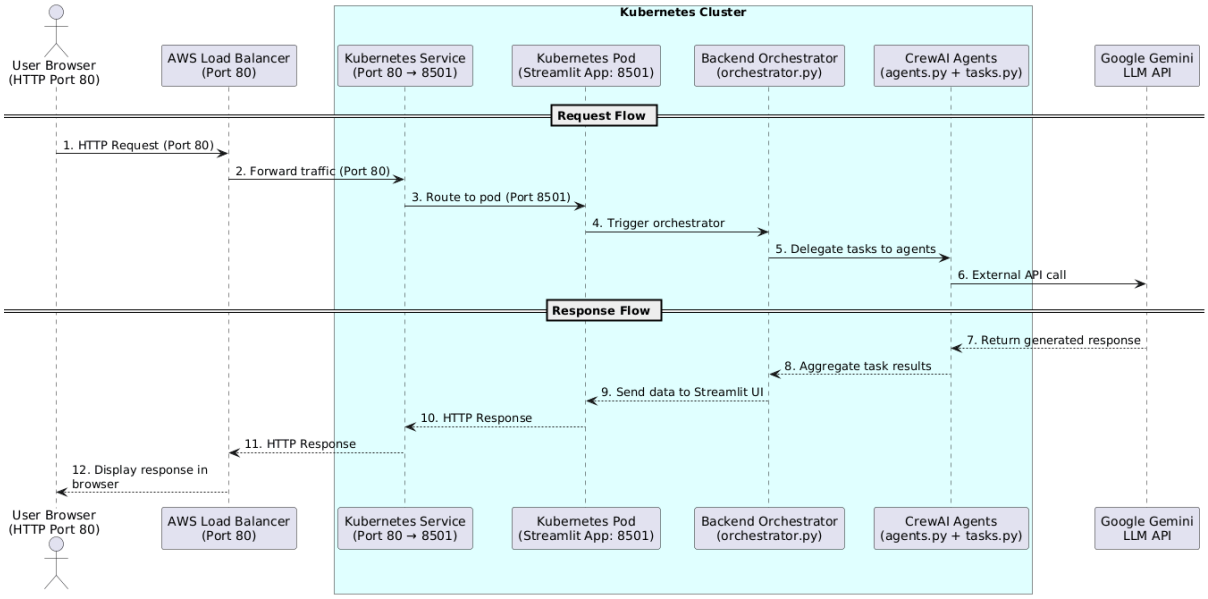
The end-to-end process flow from code change to live deployment is as follows:



2.3 Information Flow

The information flow describes how data moves through the system at runtime, from the user's browser to the AI agents and back.

User Request Flow:



2.4 Components Design

Component 1- Dockerfile

Property	Value
Base Image	python:3.11-slim
Build Strategy	Clone AI repo at build time
Working Directory	/app
Exposed Port	8501
Entry Point	streamlit run frontend/app.py
Health Check	GET /_stcore/health

Key steps: install system dependencies → copy requirements.txt → clone AI repo → install Python packages → create artifacts directory → set environment variables → define health check → set CMD.

Component 2- Kubernetes Deployment (deployment.yaml)

Parameter	Value	Reason
replicas	2	High availability
strategy.type	RollingUpdate	Zero downtime
maxSurge	1	One extra pod during update
maxUnavailable	0	No pod removed until replacement is ready
imagePullPolicy	Always	Always fetch latest image
containerPort	8501	Streamlit port
Memory Request/Limit	512Mi / 1Gi	Guaranteed and capped memory
CPU Request/Limit	250m / 500m	Guaranteed and capped CPU
Readiness Probe	/_stcore/health (delay: 30s)	Pod ready check before traffic
Liveness Probe	/_stcore/health (delay: 60s)	Restart pod if unhealthy

Component 3- Kubernetes Service (service.yaml)

Parameter	**Value**	**Reason**
type	LoadBalancer	AWS provisions public ELB automatically
port	80	Standard HTTP port for users
targetPort	8501	Forwards to Streamlit inside pod
selector	app: multi-agent-manufacturing	Routes to correct pods only

Component 4- GitHub Actions Pipeline (deploy.yml)

Step	**Action**	**Details**
1	Checkout code	actions/checkout@v3
2	Docker login	DOCKER_USERNAME + DOCKER_PASSWORD secrets

3	Build image	docker build with git clone inside
4	Push image	docker push to Docker Hub
5	AWS auth	AWS_ACCESS_KEY_ID + AWS_SECRET_ACCESS_KEY
6	Update kubeconfig	aws eks update-kubeconfig
7	Deploy	kubectl apply deployment.yaml + service.yaml
8	Verify	kubectl rollout status

Component 5- AWS Infrastructure

Component	Service	Configuration
Kubernetes Cluster	AWS EKS	Version 1.29, ap-south-1
Worker Nodes	EC2 t3.small	Min: 1, Max: 2, Desired: 2
Container Registry	Docker Hub	tiwariom/multi-agent-manufacturing
Load Balancer	AWS ELB	Auto-provisioned by K8s Service

2.5 Key Design Considerations

1. Two-Repository Separation- The AI application code and DevOps infrastructure are maintained in completely separate repositories. The Dockerfile clones the AI repo at build time using git clone, ensuring the DevOps repo remains a pure infrastructure repository with no application code. This makes both repos independently maintainable and allows the AI team and DevOps team to work without interfering with each other.

2. Clean Linux-Compatible requirements.txt- The original requirements.txt in the AI repository contained Windows-only packages such as pywin32 that cause build failures on Linux-based Docker containers. A separate clean requirements.txt was created in the DevOps repository containing only Linux-compatible dependencies, ensuring reliable and repeatable Docker builds.

3. Health-Check-Driven Rolling Updates- Streamlit exposes a built-in health endpoint at /_stcore/health. This endpoint is used as both the readiness probe and liveness probe in the Kubernetes deployment. The readiness probe ensures Kubernetes only sends traffic to a pod once Streamlit is fully initialized and ready to serve requests. The liveness probe ensures that if a pod becomes unresponsive, Kubernetes automatically restarts it.

4. maxUnavailable: 0 Guarantees Zero Downtime- Setting maxUnavailable: 0 in the rolling update strategy means Kubernetes will never terminate a running pod until its replacement has passed the readiness probe. Combined with maxSurge: 1, this ensures at least 2 healthy pods are always serving traffic during any deployment, eliminating any possibility of downtime.

5. AWS Mumbai Region (ap-south-1)- The EKS cluster is deployed in the ap-south-1 (Mumbai) region as it is geographically closest to India, providing the lowest network latency for Indian users accessing the application.

6. Runtime API Key Entry- The Google Gemini API key is never stored in the Docker image, Kubernetes manifests, or any configuration file. Users enter their API key directly in the Streamlit sidebar at runtime. This eliminates the risk of API key exposure through image inspection, repository history, or log files.

7. Cost-Conscious Infrastructure- EC2 t3.small instances are used as worker nodes to minimize AWS costs. The EKS cluster is deleted when not in use, keeping the total project cost to approximately $1–2 using AWS free credits. The t3.small instance type provides sufficient compute for running 2 Streamlit pods while remaining within budget.

2.6 API Catalogue

Although this is a DevOps project with no custom REST API, the following endpoints are relevant to the system's operation:

Internal Health Endpoints (Streamlit Built-in)

Endpoint	Method	Port	Purpose
/_stcore/health	GET	8501	Returns HTTP 200 when Streamlit is healthy and ready
/_stcore/health	GET	8501	Used by readiness probe pod receives traffic only after this passes
/_stcore/health	GET	8501	Used by liveness probe pod is restarted if this fails 3 times

Response:

HTTP 200 OK

Body: "ok"

External API Used by Application

API	Provider	Purpose	Authentication
Gemini LLM API	Google	Powers CrewAI agent intelligence	API Key entered by user at runtime via Streamlit UI

Docker Hub Registry API (used by GitHub Actions)

Action	Command	Purpose
Push Image	docker push tiwariom/multi-agent-manufacturing:latest	Upload built image to registry
Pull Image	Handled by EKS automatically	EKS pulls image on pod creation

AWS APIs (used by GitHub Actions)

API	Command	Purpose
EKS kubeconfig	aws eks update-kubeconfig	Authenticate kubectl with EKS cluster
Kubernetes API	kubectl apply -f deployment.yaml	Create/update deployment on cluster
Kubernetes API	kubectl apply -f service.yaml	Create/update LoadBalancer service
Kubernetes API	kubectl rollout status	Verify deployment completed successfully

3 Interfaces

The system has three main interfaces. The **user interface** is the Streamlit web UI accessible via the AWS Load Balancer public URL on port 80 no login required, works in any browser. The **DevOps interface** consists of GitHub for source control, Docker Hub for image storage, AWS Console for cluster management, and kubectl/AWS CLI for terminal-based control. The **inter-component interface** follows a simple chain: browser → Load Balancer (port 80) → Kubernetes Service → Pod (port 8501) → Google Gemini API (HTTPS).

4 State and Session Management

Streamlit manages all UI state in memory per browser session. Each session is completely isolated API keys and query results are never shared between users or persisted to disk. Each agent run generates a unique run ID and artifacts are written to /app/artifacts/ inside the container. Since containers are stateless, artifacts are lost on pod restart a persistent volume or S3 bucket would be used in a production system. Kubernetes itself maintains desired state via the ReplicaSet if a pod crashes, it is automatically recreated to maintain the declared replica count of 2.

5 Caching

The Dockerfile is structured to maximize Docker's layer cache. System dependencies and pip packages are installed before the git clone step so if only the AI repo code changes, Docker reuses all cached layers and skips straight to cloning. This reduces rebuild time significantly. imagePullPolicy: Always is set in the deployment manifest to ensure Kubernetes always pulls the latest image on every deploy, bypassing node-level cache. No application-level caching is implemented every agent run makes a fresh live call to the Gemini LLM API, which is intentional as supplier data must always be current.

6 Functional Requirements

6.1 User Authentication and Authorization

There is no login system. Users access the app via the public Load Balancer URL and provide their Google Gemini API key through the Streamlit sidebar. The key is injected into the environment at runtime and never written to disk, committed to any repository, or included in the Docker image. The query form remains hidden until a valid API key is entered.

6.2 Infrastructure Configuration Management

All infrastructure is version controlled in the DevOps GitHub repository. Kubernetes manifests define the desired cluster state declaratively. Credentials are stored as GitHub Secrets and injected at pipeline runtime never hardcoded in any file. The base image and package versions are pinned for reproducible builds.

6.3 Provisioning Control and Monitoring

EKS Auto Mode automatically provisions and deprovisions EC2 nodes based on pod demand. Pod health is monitored via Readiness and Liveness probes on Streamlit's built-in /_stcore/health endpoint. Unhealthy pods are automatically restarted. The GitHub Actions pipeline verifies each deployment with kubectl rollout status before marking the run as successful.

6.4 Dockerfile (Application Containerization)

The Dockerfile uses python:3.11-slim as the base image and clones the AI application repository at build time using git clone. A clean Linux-compatible requirements.txt is maintained in the DevOps repository to override the Windows-only pip freeze from the AI repo. Streamlit is configured to listen on 0.0.0.0:8501 and a health check is defined on /_stcore/health to support Kubernetes probes.

6.5 GitHub Actions (Workflow Configuration)

The pipeline triggers automatically on every push to the main branch. It runs on a GitHub-hosted Ubuntu runner, builds the Docker image on GitHub's servers, pushes it to Docker Hub, authenticates with AWS using stored secrets, updates the kubectl configuration for the EKS cluster, and applies both Kubernetes manifests. A rollout status check at the end confirms the deployment completed successfully.

6.6 CI/CD Strategy

Every push to the DevOps repository triggers a full build and deploy cycle — no manual steps required after git push. The rolling update strategy (maxSurge: 1, maxUnavailable: 0) ensures the app stays live throughout every deployment. The two-repository architecture keeps AI code and infrastructure completely independent — the AI team and DevOps team can work without interfering with each other.

7 Non-Functional Requirements

- **Availability**- Rolling updates with maxUnavailable: 0 ensure the app is always accessible during deployments. Two replicas run at all times so a single pod failure never causes downtime.
- **Scalability**- EKS Auto Mode automatically adds or removes EC2 nodes based on pod demand without manual intervention.
- **Security**- No credentials exist in any file, image, or repository. The Gemini API key is entered by the user at runtime and never persisted anywhere.
- **Performance**- The EKS cluster is deployed in ap-south-1 Mumbai the closest AWS region to India minimizing latency for users.

- **Maintainability**- A single git push triggers the entire build and deploy pipeline. No manual server access or deployment commands are needed after initial setup.
- **Cost Efficiency**- EC2 t3.small instances are used to minimize compute costs. The cluster is deleted when not in use, keeping total project spend within $1–2 using AWS free credits.

8 References

- Kubernetes Documentation https://kubernetes.io/docs
- AWS EKS Documentation- https://docs.aws.amazon.com/eks
- Docker Documentation- https://docs.docker.com
- GitHub Actions Documentation- https://docs.github.com/en/actions
- Streamlit Documentation- https://docs.streamlit.io
- CrewAI Documentation- https://docs.crewai.com
- Google Gemini API- https://ai.google.dev/docs
- Multi-agent-system Repo- https://github.com/omtiwari17/Multi-agent-system
- DevOps Repo- https://github.com/omtiwari17/Deploy-Manufacturing-App-to-Kubernetes
- Docker Hub Image- https://hub.docker.com/r/tiwariom/multi-agent-manufacturing-system

9 GitHub

https://github.com/omtiwari17/Deploy-Manufacturing-App-to-Kubernetes